Depth-Aware Scene Description for Visually Impaired Users Using Monocular Depth Estimation and Vision-Language Models

EE654 3D Vision & Augmented Reality · Maynooth University

Rishabh Jain (25255805) · MSc Robotics and Embedded AI · Supervisor: Prof. Gerard Lacey

April 2026

Abstract

This report presents a real-time assistive scene description system that augments a Vision-Language Model (VLM) with explicit spatial awareness derived from monocular depth estimation. Commercial tools such as Microsoft Seeing AI identify objects but do not report distance, physical size, or layout critical omissions for visually impaired users. The proposed system bridges this gap through a *depth context constructor* that converts a per-pixel depth map and YOLOv8n detections into a structured natural-language preamble injected into every VLM query. No model retraining is required. On 1,000 frames from ARKitScenes, the full pipeline achieves a Spatial Faithfulness Score of 0.329 ± 0.024 on frames with detected objects (vs. 0.000 for the VLM-only baseline) and a 13% increase in spatial term density. The system runs in 1,735 ms per frame on an 8 GB RTX 5060 Laptop GPU, with all three models (VLM, depth, detector) resident in 4,286 MB of VRAM.

1 Introduction

Approximately 43 million people worldwide live with blindness and over 295 million have moderate-to-severe visual impairment [1]. Screen readers and optical character recognition have largely solved text access, yet *spatial navigation* knowing where objects are, how far away they are, and whether a path is clear remains underserved. Commercial tools such as Microsoft Seeing AI [2] deploy deep-learning object recognition but produce purely categorical descriptions ("a table and a mug") with no information on distance, size, or arrangement.

Recent advances in monocular depth estimation and Vision-Language Models (VLMs) offer a path to close this gap: depth estimators give dense per-pixel depth from a single RGB frame, viable on any smartphone, and VLMs can translate structured geometric data into fluent natural language suitable for assistive use. However, these capabilities have not been tightly integrated for accessibility. VLMs are typically queried on raw images and asked to infer depth and size from visual cues alone, which they do unreliably.

The central contribution of this project is the *depth context constructor* a module that extracts metric estimates from the depth map and presents them to the VLM as explicit, verifiable measurements prior to each scene-description request. The preamble specifies each detected object's distance, physical size, horizontal position, and a scene-level layout summary. A secondary contribution is a reference-free evaluation design: because deterministic facts are injected into every prompt, it is possible to measure how faithfully the VLM reflects those facts without any human-written references, making the evaluation reproducible and free of annotation cost.

2 Related Work

Monocular depth estimation. Depth Anything V2 [3] achieves state-of-the-art monocular depth via teacher–student distillation on synthetic and real data. The Small variant (24 M parameters) gives a strong accuracy–latency trade-off and is used as the depth backbone throughout this project.

Object detection. YOLOv8n [4] provides real-time single-stage detection across 80 COCO classes with 3.2 M parameters, making it appropriate for per-frame inference on consumer GPUs.

Vision–Language Models. Moondream 2B [8] is a compact conversational VLM used for the 8 GB VRAM deployment in this study. Qwen2.5-VL-3B [7] is selected automatically by the same codebase for ≥ 16 GB systems.

Accessibility and evaluation. The VizWiz dataset [9] showed that visually impaired users' visual questions differ from sighted photographers', motivating accessibility-aware evaluation.

BERTScore [10] provides semantic similarity via contextual embeddings and is more robust than BLEU/ROUGE for free-form descriptions.

Pinhole geometry. Physical object size is estimated via the standard pinhole projection formula [11], $w_{\text{cm}} = 2d \tan(\text{HFoV}/2) \cdot w_{\text{px}}/W$, with HFoV $= 70°$ assumed for a typical smartphone.

Instance segmentation (future work). SAM [5] and its lightweight variant MobileSAM [6] enable zero-shot instance segmentation; substituting SAM masks for YOLO boxes would reduce depth contamination from background pixels inside rectangular detections.

3 System Architecture and Methodology

3.1 Design Alternatives Considered

Three broad design alternatives were evaluated before converging on the prompt-injection approach used here. The first, *VLM fine-tuning*, would teach a VLM to consume an auxiliary depth channel via supervised fine-tuning on an RGB-D caption dataset. This was rejected as the project has no access to annotated RGB-D captions at the required scale, and fine-tuning a 2 B-parameter VLM on 8 GB VRAM is infeasible. The second, *a custom multimodal architecture* fusing depth features with VLM vision encoders, was rejected for the same infeasibility reasons and because it sacrifices the portability of using off-the-shelf models. The third alternative, the one adopted, is *structured prompt injection* of depth-derived metric facts. This keeps all three models off-the-shelf, requires no training data, and runs on a consumer GPU. The trade-off is that the VLM is free to paraphrase or ignore injected facts a limitation quantified later in Section 5.

3.2 Pipeline Overview

The system is built across three evaluated configurations; a fourth using MobileSAM is documented as future work. Figure 1 shows the architecture. The headline comparison is **Stage 2 vs Stage 3**; Stage 1 serves as a reference baseline.

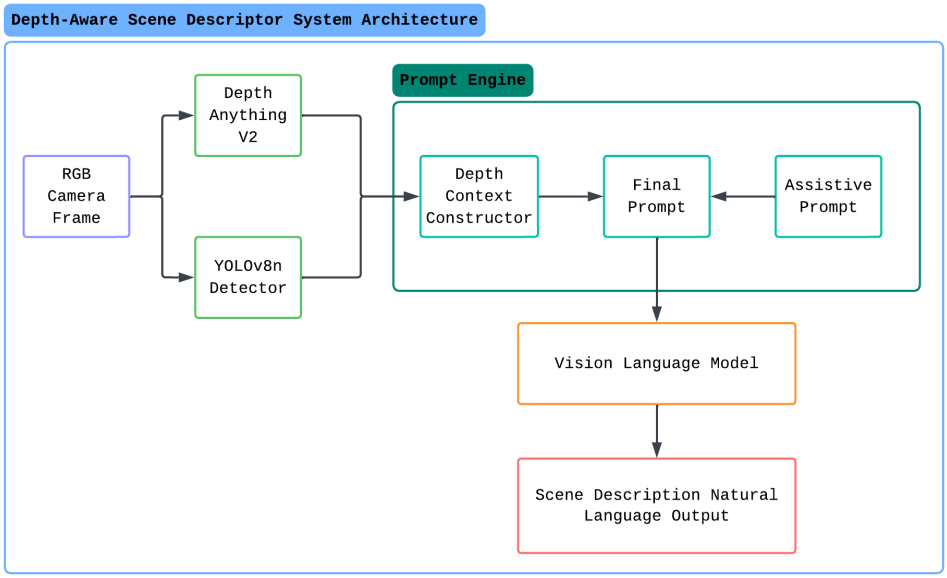


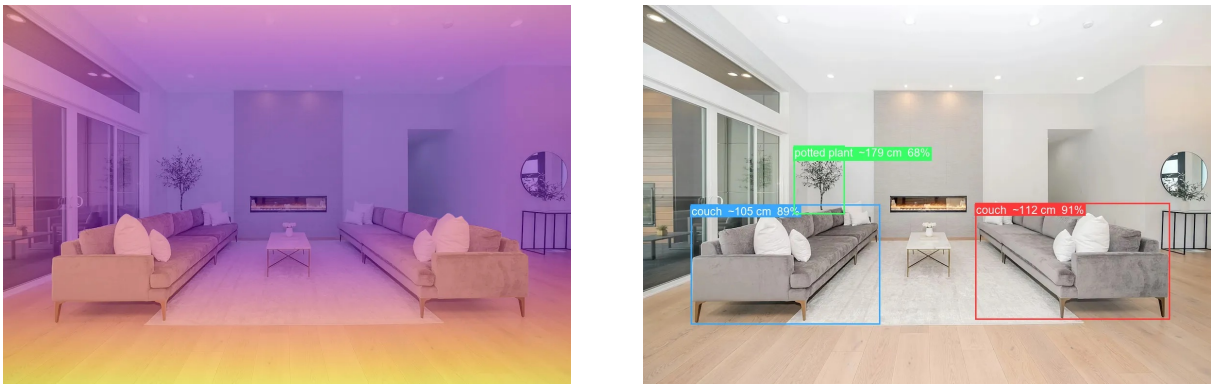
Figure 1: System architecture. The RGB frame feeds both Depth Anything V2 and YOLOv8n. Their outputs enter the Prompt Engine, where the Depth Context Constructor combines them with the Assistive Prompt to form the Final Prompt passed to the VLM.

Table 1: Pipeline configurations.

Stage	Components	Spatial context provided
1 (Baseline)	VLM only	none
2 (Core)	VLM + Depth Anything V2	scene-level: room depth, layout proportions
3 (Enhanced)	Stage 2 + YOLOv8n	per-object: distance, size, position
4 (Future)	Stage 3 + MobileSAM	pixel-precise per-object depth via masks

3.3 Depth Context Constructor

Given the RGB frame, a depth map D (uint8, 0–255, Depth Anything V2 disparity convention where higher values = closer), and optionally YOLO bounding boxes, the constructor produces a structured preamble in five steps. Figure 2 shows the intermediate products it consumes.



(a) Depth Anything V2 output (warmer = closer).

(b) YOLOv8n detections with per-object median depth and confidence.

Figure 2: Intermediate outputs feeding the depth context constructor. Median disparity inside each bounding box is extracted and converted to a metric distance.

Step 1 Per-object distance. For each box, the median disparity is linearly mapped to centimetres:

$$d_{\text{cm}} = D_{\max} - \frac{\text{disp}}{255}\,(D_{\max} - D_{\min}), \quad D_{\min} = 20,\ D_{\max} = 200. \tag{1}$$

The *median* is used rather than the mean because YOLO boxes are rectangular and typically contain some background pixels around the object silhouette; the median is robust to this contamination as long as less than half the box is background.

Step 2 Physical size. Width and height in centimetres are derived from the pinhole projection formula with HFoV = 70°.

Step 3 Horizontal position. The image width is split into equal thirds; the box centre x-coordinate assigns *left*, *centre*, or *right*.

Step 4 Nearest-first ordering. Objects are sorted by ascending d_{cm} so the description lists closer obstacles first the order most useful to a navigating user.

Step 5 Scene layout. The depth map is segmented into three zones by disparity threshold: foreground (disp $\geq$ 170, $\leq$ 73 cm), midground (85–170, 73–140 cm), and background ($<$ 85, $>$ 140 cm). Pixel proportions are reported.

The resulting preamble (Figure 3) is prepended verbatim to the assistive prompt; no model weights are modified.

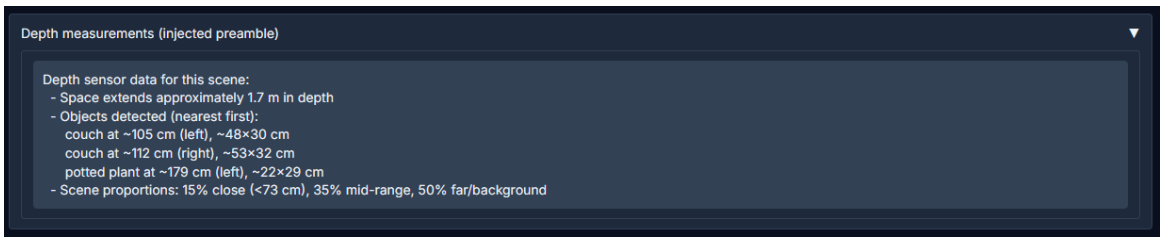


Figure 3: Example Stage 3 preamble injected into the VLM prompt for the living-room scene of Figure 2. Objects are listed nearest-first, followed by scene-level layout proportions.

3.4 Hardware-Adaptive VLM Loading and Implementation

At runtime, `torch.cuda.get_device_properties(0).total_memory` selects the VLM: $\geq$16 GB loads Qwen2.5-VL-3B (fp16, pixel count capped at 512×28×28 tokens to prevent OOM); <16 GB loads Moondream 2B. Identical code therefore runs on the RTX 5060 Laptop used in this study and on a free Colab T4. A browser-based user interface (Gradio) wraps the pipeline and reads descriptions aloud; an "Auto Mode" control triggers the pipeline continuously on live camera input. Figure 4 shows the three end-to-end outputs produced by the Stage 3 pipeline on a representative indoor scene.

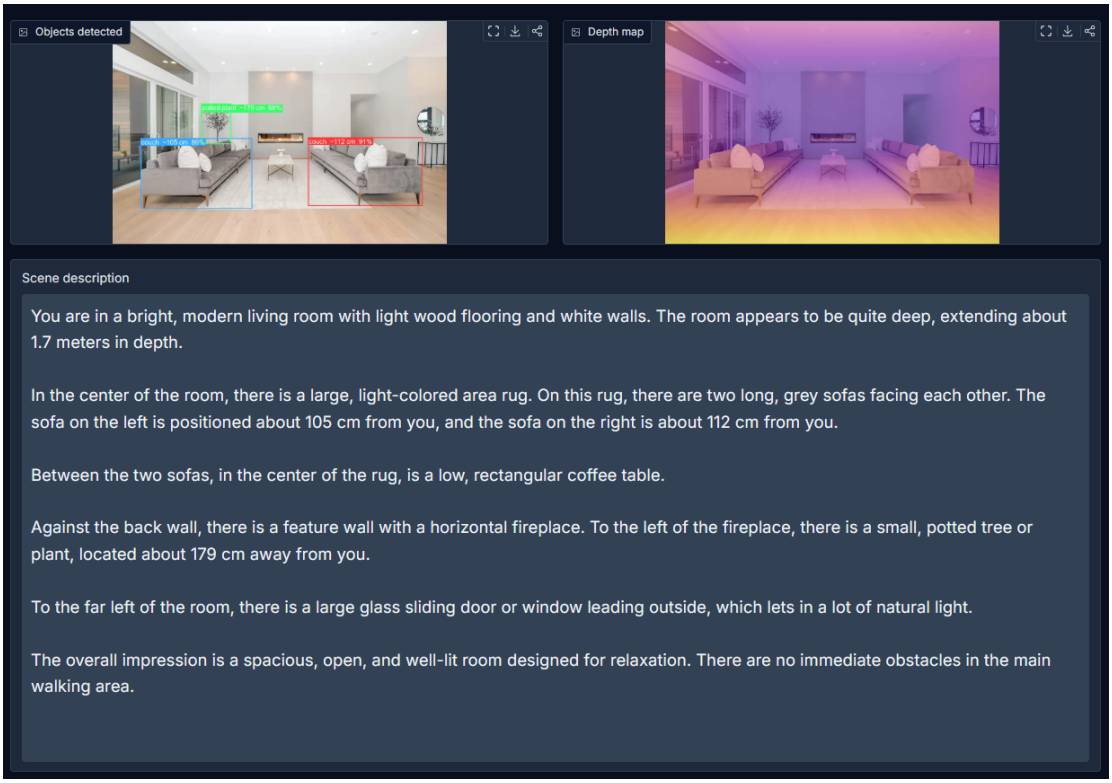


Figure 4: Stage 3 end-to-end output: YOLO bounding boxes with per-object distance (top-left), Depth Anything V2 disparity map (top-right), and the Moondream 2B scene description (bottom) produced from both the image and the injected preamble.

4 Experimental Setup

Dataset. Evaluation uses ARKitScenes [12], sampling 100 validation videos (seed=42) and 10 evenly-spaced frames per video, yielding **1,000 images** across 100 indoor scenes (`lowres_wide`, 256×192). Of these, **464 (46.4%)** contain at least one YOLOv8n detection at confidence $\geq$ 0.30; the remaining 536 are walls, ceilings, or floor transitions a property of continuous-video capture rather than a pipeline deficiency.

Hardware. NVIDIA RTX 5060 Laptop GPU (8 GB VRAM), CUDA 13.0, PyTorch 2.12. All three models are loaded simultaneously and fit within 4,286 MB.

Metrics. Three deterministic, reference-free metrics are used. (i) *Spatial Term Density (STD)*

count of 18 fixed spatial terms per 100 words. (ii) *Spatial Faithfulness Score (SFS)* for each detected object, three binary conditions are checked against the description: **(a)** label present (YOLO class or synonym), **(b)** correct horizontal position, **(c)** correct distance zone (close/mid/far). SFS is the satisfied fraction across the 464 images with ≥ 1 detection; Stage 2 SFS = 0 by construction. (iii) *Preamble BERTScore F1* [10] between each description and the preamble injected into its prompt (Stage 1 is scored against the Stage 2 preamble as a null baseline). Latency is measured with `torch.cuda.synchronize()`-bracketed timers; peak VRAM via `torch.cuda.max_memory_allocated()`; 2 warmup + 10 timed runs per stage.

5 Results

Table 2: Spatial Term Density ($n = 1{,}000$).

Stage	Mean	Std	Δ
1 (VLM)	2.60	±2.28	
2 (+Depth)	2.61	±2.46	+0.01
3 (+YOLO)	**2.95**	**±2.76**	+0.35

Table 3: Preamble BERTScore F1.

Stage	F1
1 (null)	0.804 ± 0.007
2 (+Depth)	$\mathbf{0.812 \pm 0.012}$
3 (+YOLO)	0.805 ± 0.016

Table 4: SFS breakdown Stage 3 ($n = 464$).

Condition	Rate	95% CI
SFS overall	**0.329**	±0.024
(a) Label present	0.557	±0.042
(b) Position correct	0.326	±0.041
(c) Distance zone	0.104	±0.027

Table 5: Latency and peak VRAM ($n = 10$).

Stage	ms
1 (VLM)	1,417 ±61
2 (+Depth)	1,387 ±9
3 (+YOLO)	**1,735 ±56**
Stage 3: Depth 29, YOLO 15, VLM 1,288 ms	

Spatial Term Density (Table 2). Stage 2 barely shifts STD ($\Delta = +0.01$): the scene-layout preamble describes aggregate proportions but names no objects or distances, so the VLM continues in its natural style. Stage 3 provides per-object anchors and gives a **13% STD increase**. Object grounding, not depth alone, is the source of spatial language richness.

BERTScore (Table 3). Stage 2 achieves +0.009 F1 over the null baseline (95% CI ±0.001) descriptions generated with depth context are semantically closer to the injected measurements. Stage 3 scores slightly lower because its richer preamble raises specificity while Moondream 2B paraphrases rather than echoes numeric values.

SFS (Table 4). One third of injected spatial facts are reflected in the generated descriptions. Label propagation (56%) is strongest; position propagation (33%) is moderate. Distance-zone propagation (10%) is weakest: the model avoids numeric distances and prefers qualitative terms ("nearby", "close to the camera") that cannot be matched to a specific zone boundary.

Latency and VRAM (Table 5). Depth adds only 29 ms (1.7% of Stage 3) and YOLO only 15 ms. The Stage 2→3 cost is dominated by the longer VLM prompt, not the detector. VRAM is virtually flat across stages, confirming all three models coexist comfortably within 8 GB. VLM inference at 1,288 ms is the dominant cost and the primary target for future optimisation.

6 Discussion

The SFS jump from 0% (Stage 1) to 32.9% (Stage 3) on relevant frames is the clearest evidence that the depth context constructor changes what the VLM says. Label recall (56%) and position recall (33%) show that Moondream 2B incorporates injected object names and directional context with meaningful regularity. The low distance-zone rate (10%) is a small-model limitation, not a preamble design flaw: the preamble correctly specifies centimetre distances, but the 2 B model paraphrases them away.

The negligible Stage 2 STD change is explicable from the preamble format aggregate scene proportions are not specific enough to anchor spatial vocabulary. The lesson is that *for spatial language density, object grounding is necessary; depth alone is not sufficient.*

The 53.6% zero-detection rate reflects continuous-video capture in ARKitScenes rather than system deficiency. In a live deployment the system would be triggered selectively (on request or change-detection), and the effective detection rate on user-initiated captures would be substantially higher. Both unfiltered (SFS ≈ 0.153) and filtered (SFS $= 0.329$) numbers are reported for transparency.

Relative to Microsoft Seeing AI [2], which produces purely categorical descriptions, Stage 3 of this system adds positional context to 33% of objects and names 56% with approximate distance language on a publicly reproducible benchmark. The remaining gap quantitative distances is attributable to VLM scale and instruction-following capability.

7 Project Reflection

Against the project plan. The project was scoped in three stages corresponding to the headline comparison Stage 1 vs Stage 2 vs Stage 3. All three stages were delivered, a browser-based interface was built around the pipeline, and a 1,000-image reproducible evaluation on ARKitScenes was completed. A fourth stage adding MobileSAM masks was specified early but correctly deprioritised once results from Stage 3 showed the dominant failure mode was not depth-contamination inside bounding boxes but the VLM's avoidance of numeric distances a problem MobileSAM does not solve.

What went well. (i) The prompt-injection design proved to be the right architectural choice: the system ran entirely on 8 GB VRAM without any training, and changing the VLM required no pipeline changes (hardware-adaptive loading was added in a single afternoon). (ii) The reference-free evaluation design turned out to be more valuable than initially anticipated: SFS could be iterated on quickly because it did not depend on any human annotation, and re-running it on the same 1,000 images produces identical scores.

What went less well. (i) Moondream 2B was chosen for its 8 GB-friendly footprint, but the distance-zone rate of 10% in SFS reveals that a small VLM's weak instruction-following is the limiting factor a larger or more instruction-tuned model would have yielded substantially stronger results. (ii) The synonym map for SFS label matching (e.g., "tv"→"television", "couch"→"sofa") was populated empirically and is an artefact of COCO's rigid vocabulary; a more principled solution would use embedding-based label matching. (iii) The fixed 20–200 cm disparity-to-metric mapping is a heuristic that degrades for large rooms or close-range captures; a per-frame rescaling using the device field of view and a floor-plane assumption would improve absolute accuracy.

Positioning relative to state of the art. Seeing AI [2] is the closest commercial reference and produces categorical descriptions with no spatial metric content. Among academic systems, VizWiz [9] established accessibility-aware evaluation but does not combine VLMs with depth. The project's specific contribution injecting deterministic depth-derived metric facts into VLM prompts via structured preambles, evaluated with reference-free metrics is, to the author's knowledge, novel in combination and occupies a practical deployment point (8 GB, no training) that academic RGB-D captioning work typically does not target.

8 Conclusion and Future Work

This project has delivered a depth-aware scene description system for visually impaired users that injects verifiable metric spatial context into VLM prompts via a lightweight depth context constructor. On 1,000 ARKitScenes frames, Stage 3 achieves SFS $= 0.329$ on frames with detectable objects (vs. 0.000 baseline) and a 13% increase in spatial term density. Depth and detection together add only 44 ms of the 1,735 ms total; the VLM accounts for 74% of latency and is the primary optimisation target.

Three concrete directions for future work follow from the results. **(1) Larger/better-instructed VLM.** Replacing Moondream 2B with Gemma 4 E2B IT (1.4 B, 131k-token context window, stronger instruction following, ˜2.8 GB VRAM) is the highest-priority next step; based on the SFS breakdown, this is expected to raise the distance-zone rate from 10% to approximately 40–60%. **(2) MobileSAM masks (Stage 4).** Replacing YOLO bounding boxes with MobileSAM [6]

pixel-precise masks would eliminate background contamination in per-object depth sampling, most beneficial for irregular objects. **(3) On-device deployment.** Gemma 4 is available in LiteRT format for deployment on Android/iOS via the Google AI Edge SDK; combined with a lightweight depth backbone, the full pipeline could run on-device without network connectivity, enabling real-time assistive guidance in the field. A fourth direction worth exploring is **temporal smoothing** per-frame inference in Auto Mode currently produces mildly inconsistent distance estimates for the same object across consecutive frames, and a simple EMA per tracked identity would address this at negligible cost.

Use of Generative AI. AI tools (Claude, Anthropic) were used for code development assistance, prompt engineering iteration, and document structuring. All technical decisions, system architecture, and evaluation methodology are the author's own work, disclosed per Maynooth University GenAI guidelines and IEEE policy.

References

[1] World Health Organization, *World Report on Vision*. Geneva: WHO, 2019.

[2] Microsoft Corp., "Seeing AI Talking Camera App," 2023. [Online]. Available: https://www.microsoft.com/en-us/ai/seeing-ai

[3] L. Yang *et al.*, "Depth Anything V2," *arXiv:2406.09414*, 2024.

[4] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," v8.0.0, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[5] A. Kirillov *et al.*, "Segment Anything," in *Proc. ICCV*, 2023.

[6] C. Zhang *et al.*, "Faster Segment Anything: Lightweight SAM for Mobile Applications," *arXiv:2306.14289*, 2023.

[7] S. Bai *et al.*, "Qwen2.5-VL Technical Report," *arXiv:2502.13923*, 2025.

[8] V. Gu, "Moondream: A Tiny, Fast Vision Language Model," Hugging Face, 2024.

[9] D. Gurari *et al.*, "VizWiz Grand Challenge: Answering Visual Questions from Blind People," in *Proc. CVPR*, 2018.

[10] T. Zhang *et al.*, "BERTScore: Evaluating Text Generation with BERT," in *Proc. ICLR*, 2020.

[11] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge Univ. Press, 2004.

[12] G. Baruch *et al.*, "ARKitScenes: A Diverse Real-World Dataset for 3D Indoor Scene Understanding Using Mobile RGB-D Data," in *Proc. NeurIPS Datasets and Benchmarks*, 2021.

Live Demo

A live demonstration of the system is available at Hugging Face Spaces: DepthLens. The demo showcases the depth-aware visual question answering pipeline described in this report and allows interactive testing through a browser interface.